

NAME : UDHAYARAJAN J.
SUB : DBMS
CLASS : MCA-I
ROLL NO : 2202561
TYPE : UNIT - 4 ASSIGNMENT.
DATE : 2025-12-04

Q1) Explain ACID properties in details with suitable examples.

ACID properties are set of rules that ensure reliable and consistent execution of database transaction.

ACID stands for Atomicity, Consistency, Isolation, Durability. These properties ensure that even in case of failures the data remains accurate and trust worthy.

Atomicity :

Atomicity means a transaction must be treated as a single, indivisible unit. Either all operations in the transaction must execute or none should execute.

Example :

A bank transfer from Account A to Account B involves two steps:

1. Deduct ₹ 500 from A.
2. Add ₹ 500 to B.

If the system fails after step 1 but before step 2, Atomicity ensures both steps are rolled back, so no partial transaction occurs.

Consistency :

Consistency ensures that a transaction bring the database from one valid state to another valid state following all rules, constraint and triggers.

Example :

If a table has a constraint that $\text{balance} \geq 0$, a transaction that withdraws money must not violate this rule.

If someone tries to withdraw more money than available, the transaction will fail, keeping the database consistent.

Isolation :

Isolation ensures that concurrent transactions do not interfere with each other. The final result must be the same as if transactions were executed one after the other (serially).

Example :

Two users trying to purchase the last product at the same time.

T1 reads $\text{stock} = 1$

T2 also reads $\text{stock} = 1$.

Isolation ensures that only one transaction updates the stocks successfully, preventing incorrect result like $\text{stock} = -1$.

Durability :

Durability ensures that once a transaction is committed, its result are permanently saved, even in case of power failure, system crash or restart.

Example :

If a bank transfer is completed and the system commits it, the updated balance remains stored even if the system crashes immediately after.

Q2). Differentiate between Conflict Serializability and view Serializability.

Explain whether following schedule is conflict serializable or not?

using precedence graph:

T ₁	T ₂
R(A)	
R(B)	
	R(B)
W(A)	
W(B)	

Conflict Serializability :

Conflict serializability ensures database consistency by checking if a non-serial schedule can be rearranged into a serial order by swapping non-conflicting operations.

It prevents conflicting operations (like read/write on the same data) from executing at the same time.

Conflicting operations.

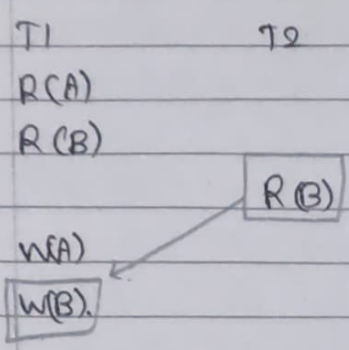
- i) R - W
- ii) W - R
- iii) W - W

view serializability:

A schedule is view serializable if it is view equivalent to some serial schedule.

view serializability ensures that a non-serial schedule results in the same final outcome as a serial schedule, maintaining database consistency.

It checks same reads, same read from and same final writes.



Conflict:

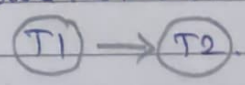
$T_2 : R(B) \rightarrow T_1 : W(B)$

They access the same data item (B) and it's a Read-write Conflict.

precedence graph:

Node : T1, T2

Edge found: $T_1 \rightarrow T_2$



No cycle. the schedule is conflict serializable.

Q2). What is Two-phase locking (2PL)? Explain Basic 2PL and Strict 2PL with example:

Two-phase locking (2PL):

Two-phase locking (2PL) is a concurrency control protocol used in DBMS to ensure serializability of transaction.

It controls how transactions acquire and release locks on data item.

2PL divides the transaction into two distinct phases:

i) Growing phase.

- Transaction can acquire locks.
- cannot release any lock.

ii). Shrink phase:

- Transaction release locks.
- cannot acquire new locks

This guarantee that once a transaction releases a lock, it will not request any more locks, ensuring conflict serializability.

Basic (2PL):

In Basic 2PL a transaction follow both phases:

Growing phase → only acquire locks.

Shrink phase → only release locks.

There is no restriction on when locks must be released.

ex:

Transaction (T1).

Lock - x (A).

WRITE (A).

Lock - S (B)

READ (B)

UNLOCK (A)

UNLOCK (B)

Here

- T1 takes locks on A and B in the Growing phase
- Then releases A and B in the Shrink phase

This ensures conflict serializability but does not prevent problems like cascading rollback.

Strict (2PL).

Strict 2PL is a stronger version of 2PL.

Rules:

- All exclusive (write) locks must be held until the transaction commit or aborts.
- Read locks may be released earlier.

strict 2PL prevents dirty reads and cascading rollbacks.

ex:

Transaction (T2)

Lock - x(A)

WRITE(A)

Lock - x(B)

WRITE(B)

-- cannot unlock any item until commit --

COMMIT

UNLOCK(A)

UNLOCK(B)

Here

T2 acquires locks manually,

Holds all write lock until COMMIT

This ensures no other transaction can read dirty data.

Q4). Explain Deadlock in DBMS with an example.

Deadlock:

A deadlock in DBMS occurs when two or more transactions are waiting for each other to release locks, and none of them can proceed.

As a result all the involved transactions are stuck forever unless the system intervenes.

Deadlock usually happens when:

- Each transaction locks some data item.
- Then requests another data item already locked by another transaction.
- This creates a circular waiting condition.

Deadlock prevents further progress and reduces system performance.

ex:

T1 locks (A).

T2 locks (B).

T1 request (B) (But (B) is locked by T2).

T2 request (A) (But (A) is locked by T1).

This creates a circular wait.

Both transactions are waiting forever. → Deadlock.

Deadlock Recovery:

If deadlock happens:

- Abort one transaction (victim)
- Rollback and restart.
- Release its locks.

Q5) write short notes on the following timestamp-based.

Concurrency Control methods with example.

a) Basic Timestamp ordering (TO) protocol.

b) Strict Timestamp ordering (STO) protocol.

c) Thomas' write Rule (TWR).

a) Basic Timestamp ordering (TO) protocol.

The Basic Timestamp ordering protocol ensures serializability by ordering all transactions based on their timestamps.

Each data item maintains two timestamps:

- $R_TS(x)$: Largest Timestamp of any transaction that successfully read (x) .

- $W_TS(x)$: Largest Timestamp of any transaction that successfully wrote (x) .

ex:

T_1 ($TS = 5$) writes $x \rightarrow W_TS(x) = 5$

T_2 ($TS = 3$) tries to write $x \rightarrow 3 < 5 \rightarrow T_2$ is roll back because it's too old.

b) Strict timestamp ordering (STO) protocol.

Strict (TO) is a stricter version of basic (TO) it ensures conflict serializability and prevents cascading roll backs.

- For every read or write timestamp ordering rules must be followed.

- All writes are delayed until it is certain that no younger transaction has read the old value.

- writes are applied only when safe, preserving strict execution order.

ex:

- T_1 ($TS=10$) writes (X) but the write is not immediately applied.
- System checks if any younger transaction ($TS > 10$) has read (X).
- If none have $w_TS(X)$ is updated, else T_1 waits or is rolled back.

c) Thomas write rules (TWR)

Thomas' write rule is a relaxation of Basic TO. It ignores certain outdated writes to improve performance.

- If a transaction T wants to write (X) but
- $TS(T) < w_TS(X) \rightarrow$ The write is obsolete and can be safely ignored (not rolled back). This reduces unnecessary rollbacks.

ex:

$w_TS(X) = 15$ (written by T with Timestamp 15)
 T_2 ($TS = 8$) tries to write x
 Since $8 < 15$ T_2 's write is ignored, not executed, and the transaction continues normally.